

Fitting Models to Data:

What it means to rely on Scipy's `curve_fit` package

```
scipy.optimize.
```

```
curve_fit
```

```
curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False,  
check_finite=None, bounds=(-inf, inf), method=None, jac=None, *,  
full_output=False, nan_policy=None, **kwargs)
```

[\[source\]](#)

[Use non-linear least squares to fit a function, f, to data.]

First, what is *linear* least squares?

numpy.linalg.lstsq

`linalg.lstsq(a, b, rcond=None)`

Return the least-squares solution to a linear matrix equation.

Computes the vector x that approximately solves the equation $a @ x = b$.

These methods can only apply when the model to fit the data is expected to be a linear combo of parameters, e.g.:

$$y = mx + b$$

or

$y = ax^2 + bx + c$ (polynomials are linear combos!)

scipy.optimize.

lsq_linear

`lsq_linear(A, b, bounds=(-inf, inf), method='trf', tol=1e-10, lsq_solver=None, lsqr_tol=None, max_iter=None, verbose=0, *, lsqr_maxiter=None)`

[\[source\]](#)

Solve a linear least-squares problem with bounds on the variables.

Given a m -by- n design matrix A and a target vector b with m elements, `lsq_linear` solves the following optimization problem:

```
minimize 0.5 * ||A x - b||**2
subject to lb <= x <= ub
```

This optimization problem is convex, hence a found minimum (if iterations have converged) is guaranteed to be global.

Okay, so what is *non-linear* least squares?

scipy.optimize.

least_squares

```
least_squares(fun, x0, jac='2-point', bounds=(-inf, inf), method='trf',  
ftol=1e-08, xtol=1e-08, gtol=1e-08, x_scale=None, loss='linear',  
f_scale=1.0, diff_step=None, tr_solver=None, tr_options=None,  
jac_sparsity=None, max_nfev=None, verbose=0, args=(), kwargs=None,  
callback=None, workers=None) \[source\]
```

Solve a nonlinear least-squares problem with bounds on the variables.

Given the residuals $f(x)$ (an m -D real function of n real variables) and the loss function $\rho(s)$ (a scalar function), `least_squares` finds a local minimum of the cost function $F(x)$:

```
minimize F(x) = 0.5 * sum(rho(f_i(x)**2), i = 0, ..., m - 1)  
subject to lb <= x <= ub
```

The purpose of the loss function $\rho(s)$ is to reduce the influence of outliers on the solution.

Broader functions can be used to describe the data, e.g.
 $y = ae^x$

But such solutions can't be found analytically; we can only test many parameters until we find something that *minimizes the distance* between the data and our models.

Well . . . can we just do that ourselves?

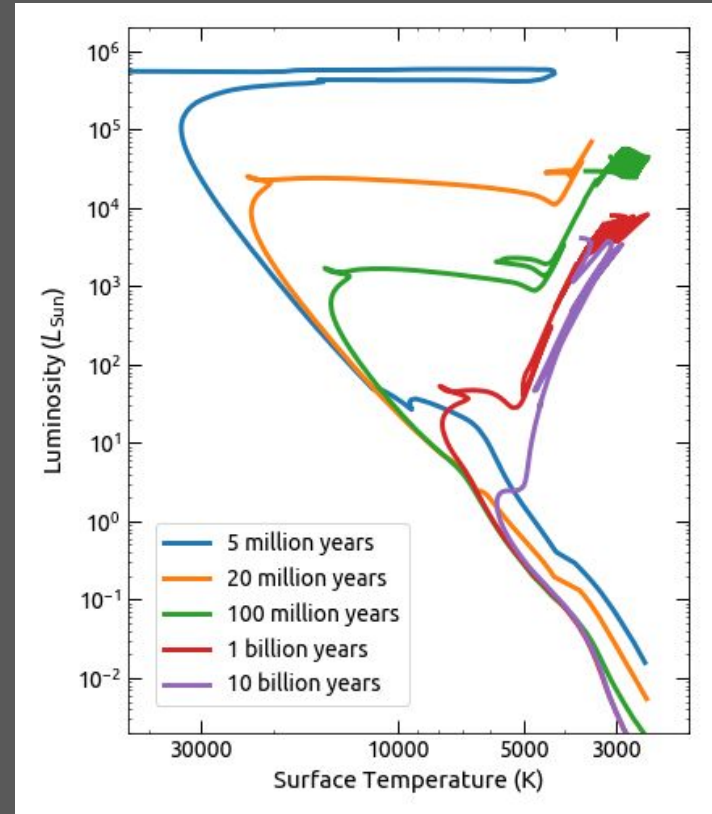
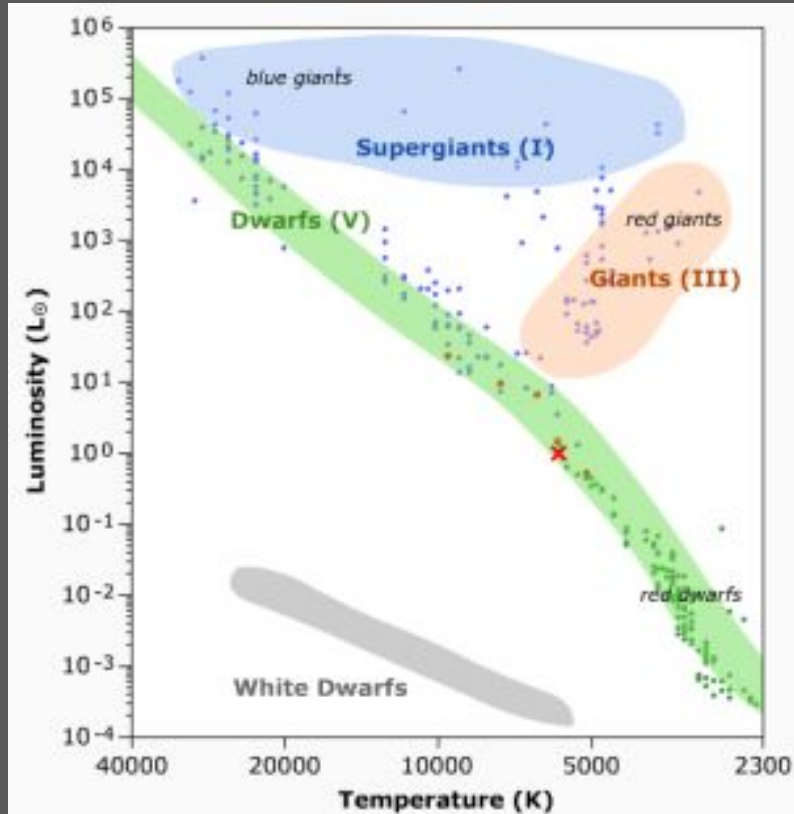
Yes, of course!

But still, `curve_fit` is right there to do it fast for us; why wouldn't we use it all the time?

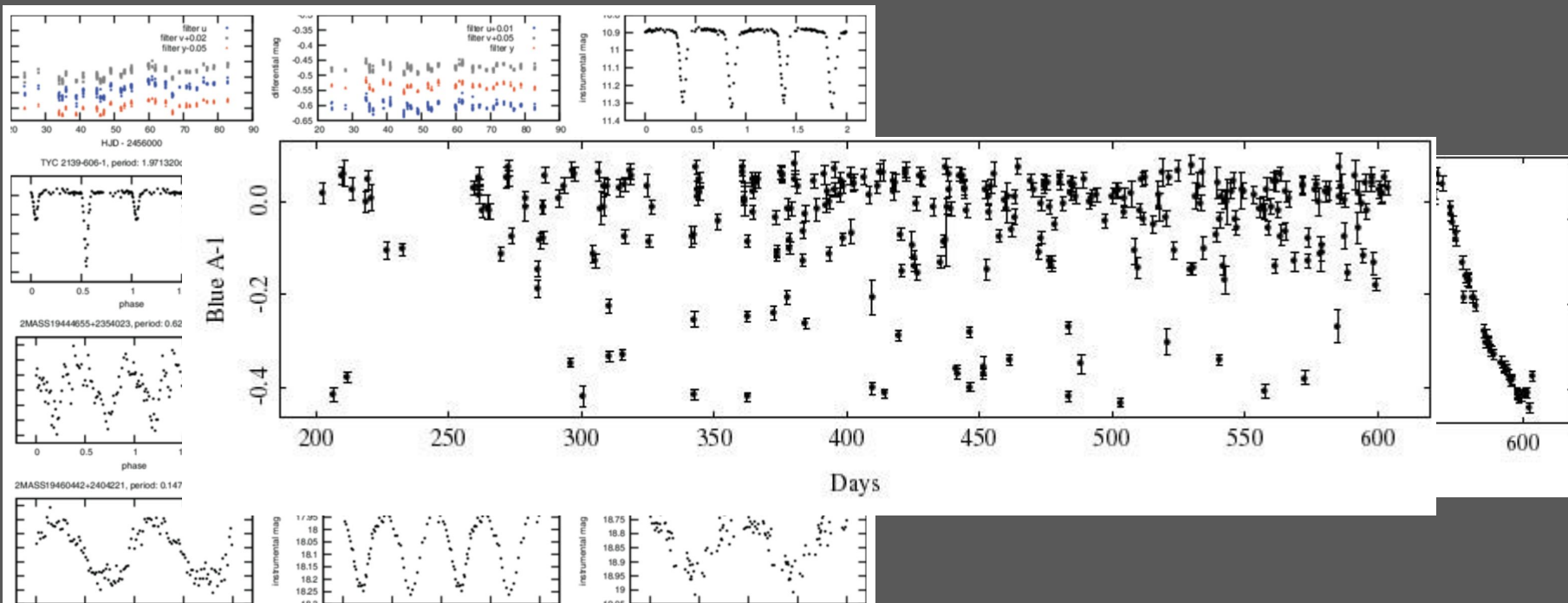
Well, what if your “models” aren't callable functions?
What if you have simulation results - discrete datasets of simulated x and y - and you just want to see which simulation fits your real x and y ?

This is today's notebook!

Finally, some background on the science in today's notebook:

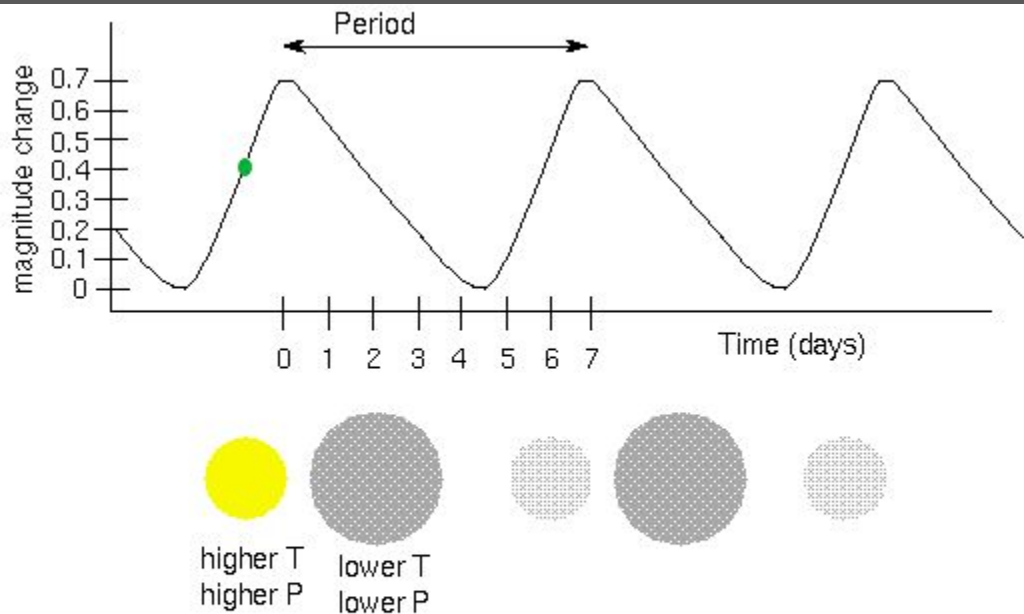


Models within models



Resulting Light Curves Of The Variable Stars - Rotating Variable Stars Light Curve

Luminosity

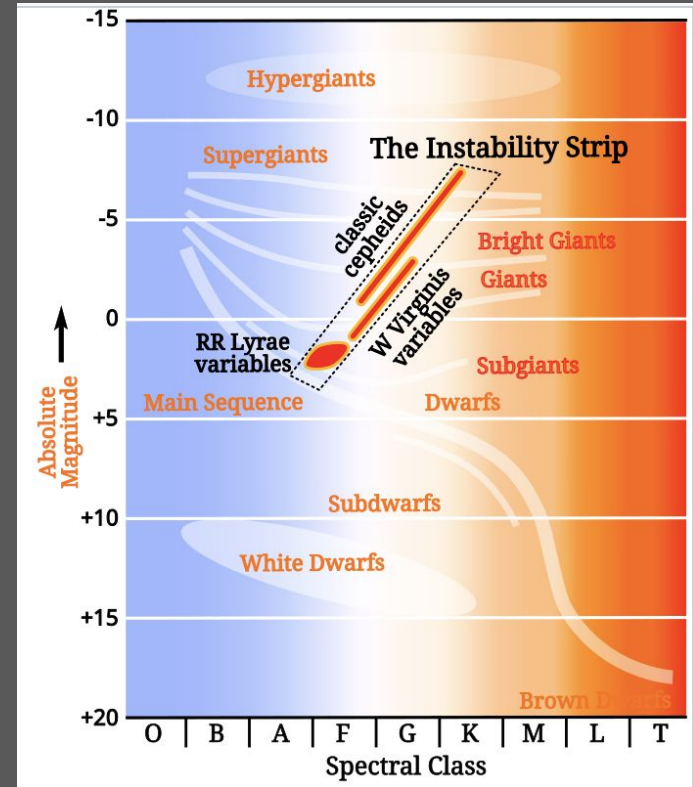


Cepheid variables: outward pressure (P) and inward gravity compression are out of sync, so star changes size and temperature: it **pulsates**.
RR-Lyrae variables are smaller and have pulsation periods of less than 24 hours. Also, their light curve looks different from the *Cepheid* light curve.

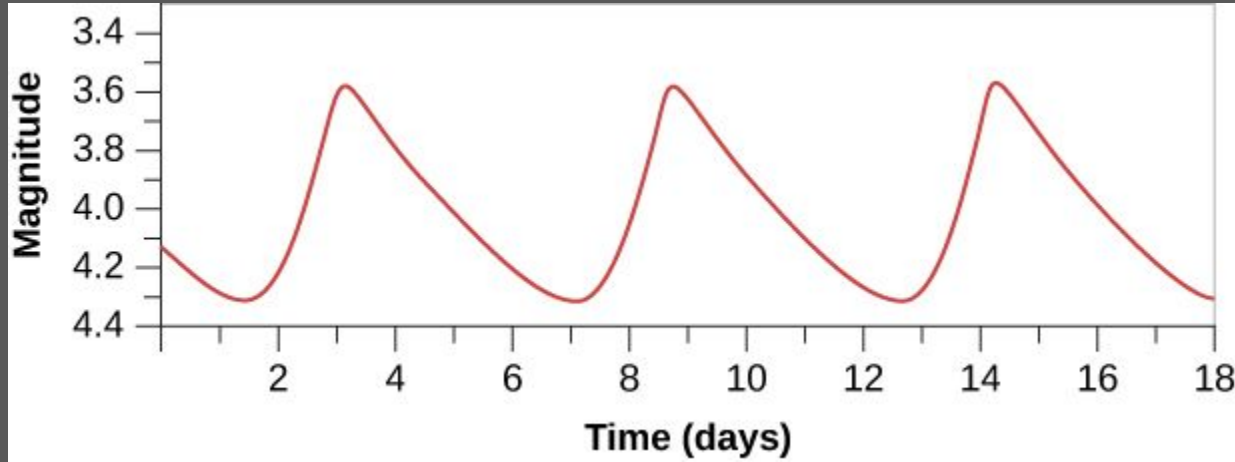
First confirmed RR Lyrae star was discovered by Williamina Fleming

Cepheid period-luminosity relationship was discovered by Henrietta Leavitt

The mass and elemental makeup of stars determine whether they become RR Lyrae, Cepheid Type 1, or Cepheid Type 2



How do you model something periodic?

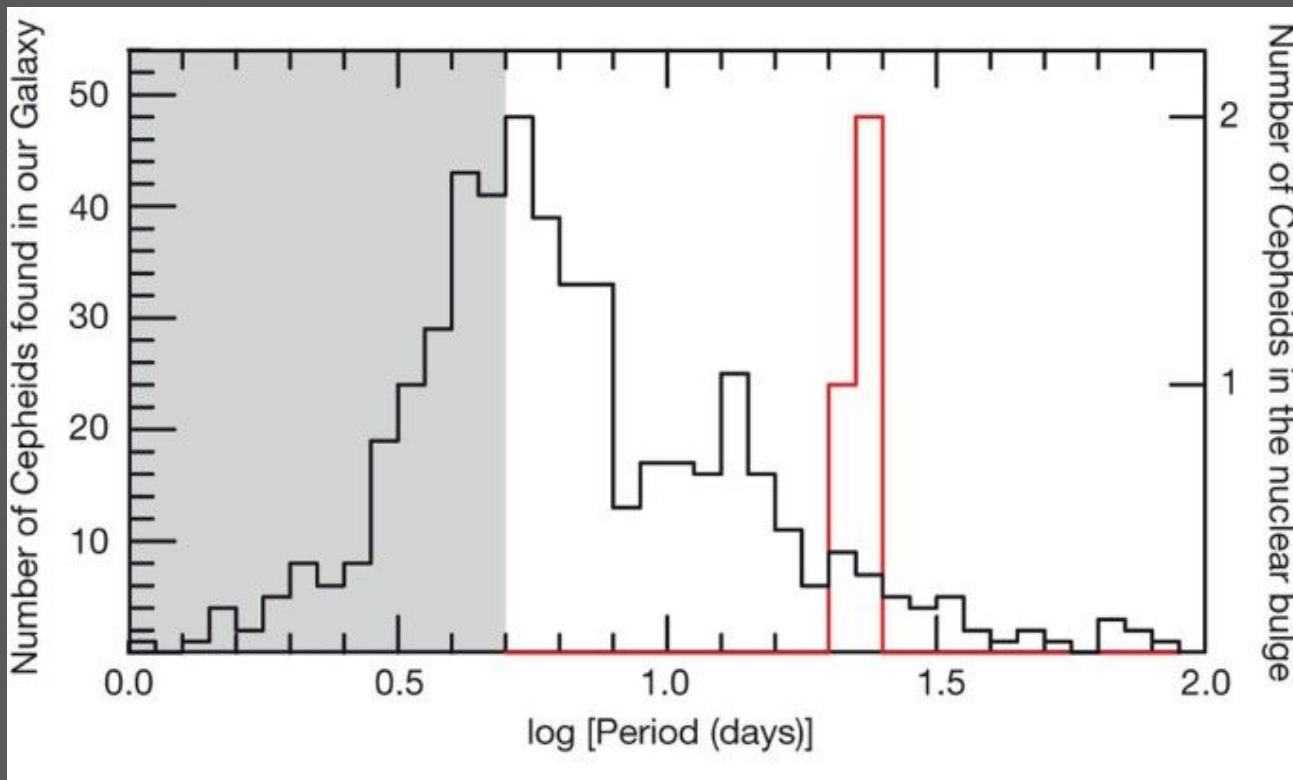


1) Functions that describe periodic things!

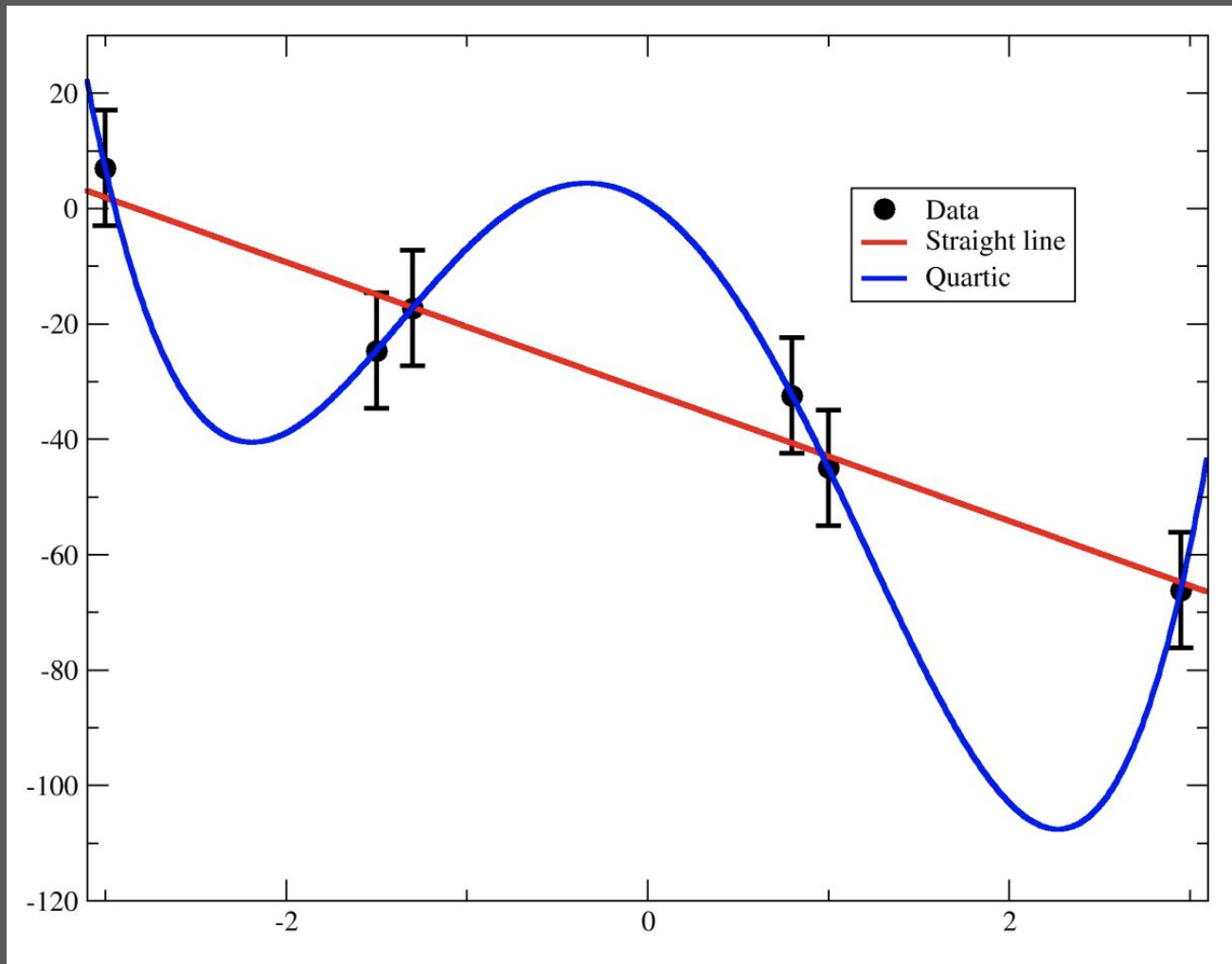
OR

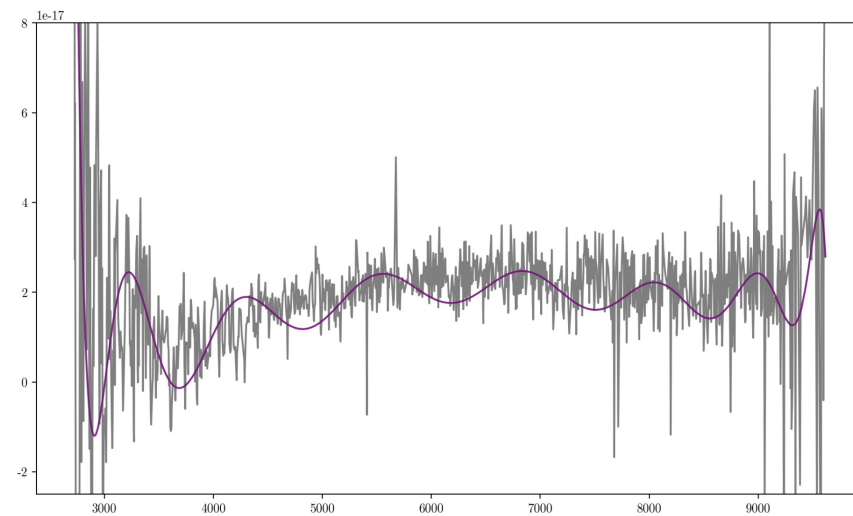
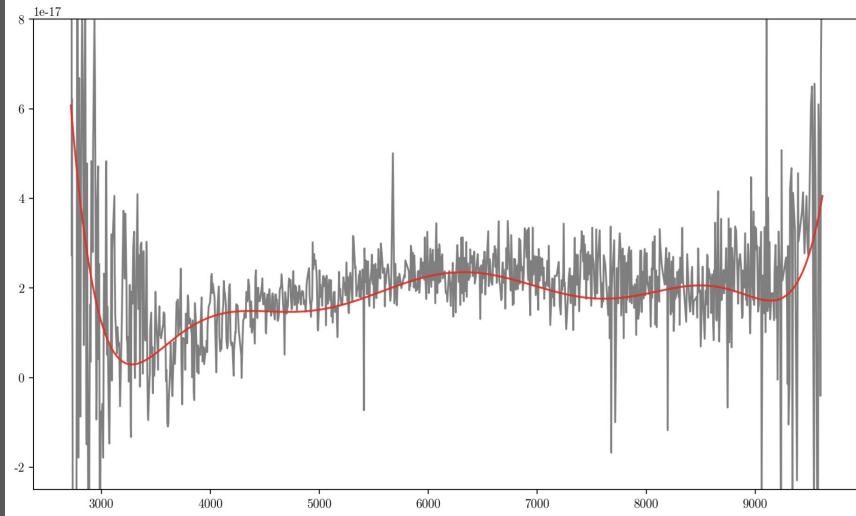
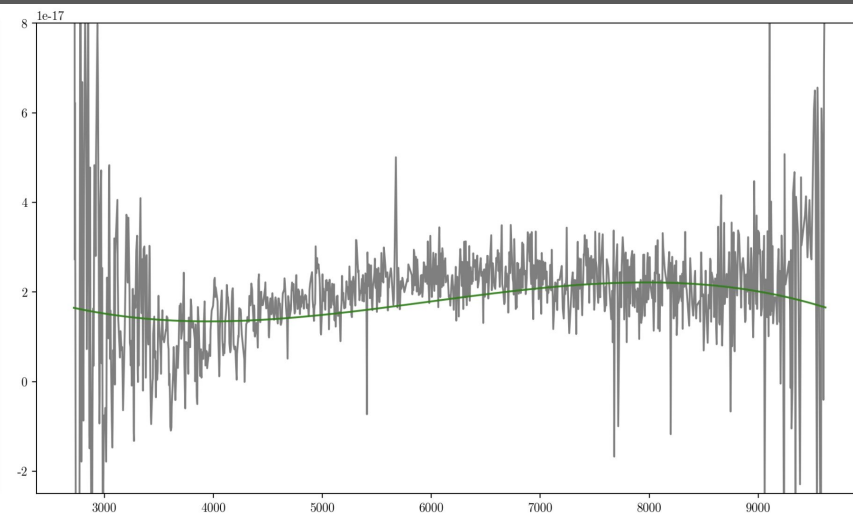
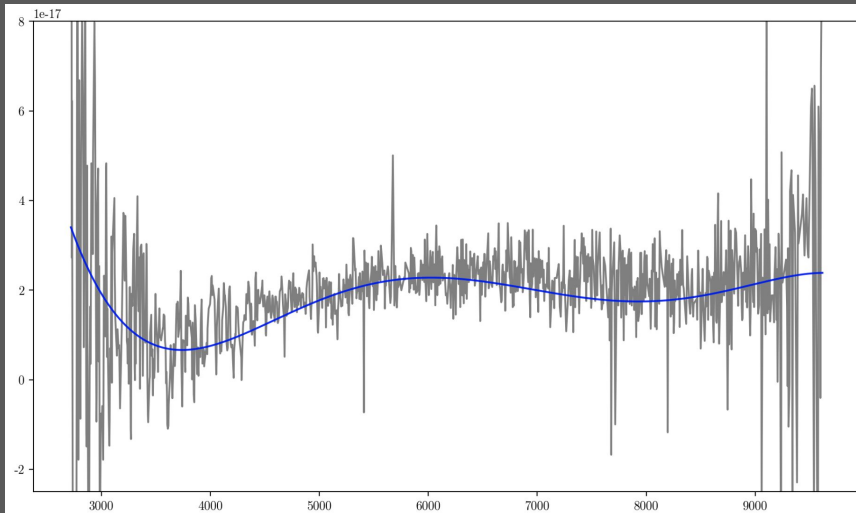
2) Use the period itself as a simplified representation of the data!

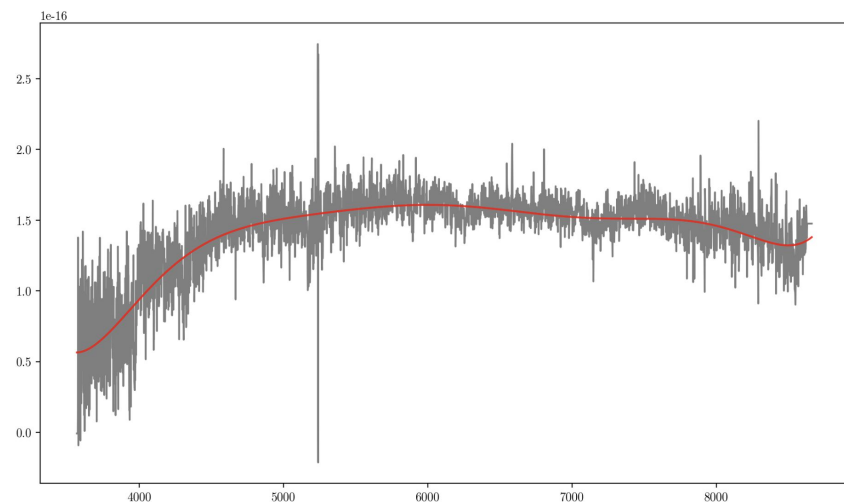
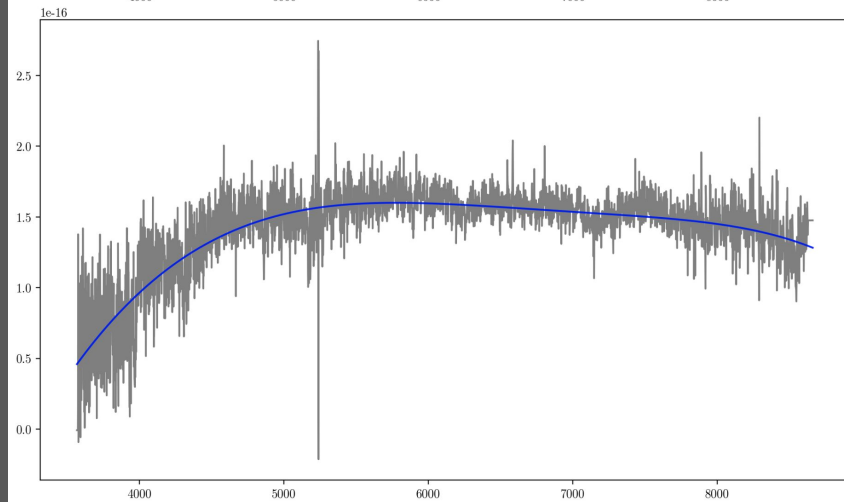
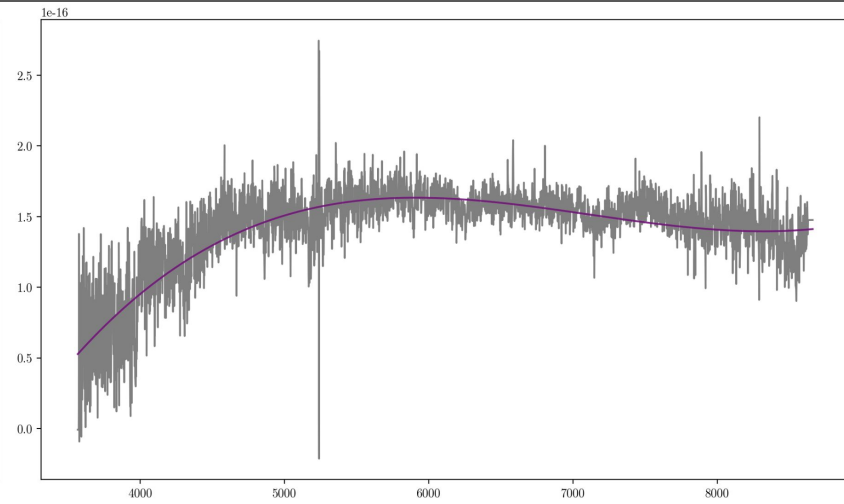
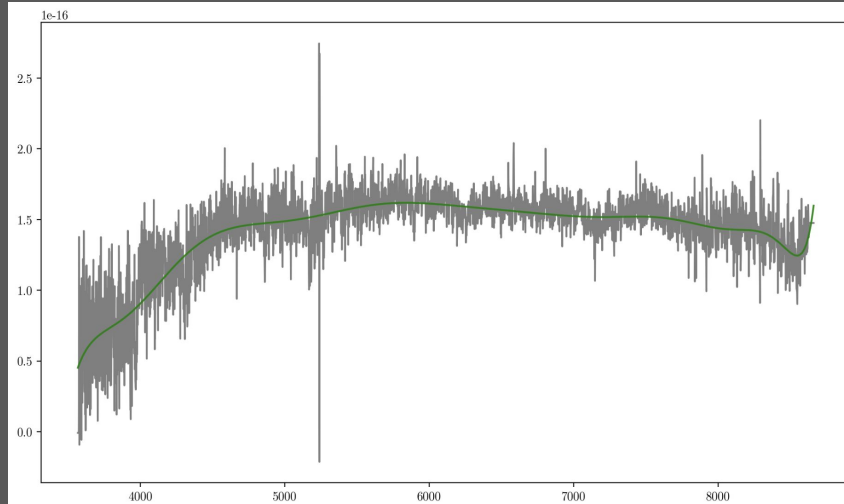
Okay, you've modeled your data, and now you have a gazillion Cepheids in a region. What can you do with *that* data?



How will
you know
when you've
overfit your
data?







- `np.polyfit` - “linear” least squares fit to data
 - For polynomials (including zero-order AKA lines)
 - You have to give the order N of the polynomial (an assumption!)
 - Solves the matrix equation directly to get all coefficients of N -order polynomial, guaranteeing the absolute “best fit” possible to your data
- `scipy.optimize.curve_fit` - non-linear least squares
 - For all functions (e^x , log, etc)
 - Will not (can not!) solve an equation, will instead minimize the difference between model and data
 - Basically minimizing Chi-Square
 - It will have to test many model parameters to come up with a “minimized” solution - it is still, somewhat, a guess!
- Can use `np.linalg.lstsq` or `scipy.optimize.least_squares` to fit your models directly without relying on other black-box parameters hard-coded into `polyfit/curve_fit`

Is there a better way???

Chi-Square

Related to a probability - the
"likelihood":

$$p(\vec{d}|\vec{p}, m) = N e^{-\chi^2/2}$$

Minimizing chi-squared is equivalent
to maximising likelihood, because the
exponential is monotonic

vs.

Bayesian

Also related to a probability:

$$p(\vec{p}|\vec{d}, m) = \frac{p(\vec{d}|\vec{p}, m) \times p(\vec{p}|m)}{p(\vec{d}|m)}$$

One of these terms should look
familiar!